Sretan EMR — Complete Hospital Setup Guide (Step by Step)

Plain-English version. Every "do this" includes HOW to do it.

This guide takes you from "nothing" to "the whole hospital is using the system and you can update it remotely from your office". Read the boxed explanations when you meet a new word — the whole guide is written so a non-technical person can follow it.

0. Understand the system in one page (read this first)

Think of a **TV station** and **TV sets**:

- **The station** = one computer ("the server box") that holds ALL the patient data and runs the program.
- **The TV sets** = the hospital's normal computers. They only *show* the program by opening a browser. They store nothing. If a TV breaks, nothing is lost.
- **The router/WiFi** = the road between the station and the TVs.
- **The cloud (Supabase)** = a small copy of your hospital data stored safely on the internet. Used so that YOU (from your office, far away) can see the hospital's status and send updates.
- **GitHub** = your online "post office" for sending new code. When you push new code there, the hospital's server can receive it automatically.

The hospital works **even if the internet is down** (offline-first). The cloud and GitHub are only needed when you push updates or when you want to watch the hospital from afar.

1. Before you buy anything — the server box specs

The "server" is **one ordinary desktop computer** that we give one job. Buy **the box only** (no need for its own monitor — you will borrow a screen once during setup).

	Minimum (works, tight)	Recommended — BUY THIS	Maximum (large/multi-hospital)
Machine type	Business desktop tower or mini PC (Dell OptiPlex, Lenovo ThinkCentre, HP ProDesk / Mini)	**Same type, newer model**	Same type, bigger
Processor (CPU)	4-core (older i3 / Ryzen 3)	**Modern 4–6 core (i5 / Ryzen 5, 12th gen or newer)**	6–8 core (i7 / Ryzen 7)
Memory (RAM)	8 GB	**16 GB**	32 GB
Storage (disk)	256 GB SSD	**512 GB NVMe SSD**	1 TB SSD

	Minimum (works, tight)	Recommended — BUY THIS	Maximum (large/multi-hospital)
Operating system	Windows 11 Pro 64-bit	**Windows 11 Pro 64-bit (already installed by the shop)**	Windows 11 Pro
Network	Built-in Ethernet port	**Built-in Gigabit Ethernet**	Gigabit
Power	—	**UPS battery 800–1000 VA (VERY important)**	UPS 1500 VA
Screen/keyboard/mouse	None needed (borrow one for setup)	None needed	None needed

Buying list example: one Dell/Lenovo/HP business desktop (i5, 16 GB RAM, 512 GB SSD, Windows 11 Pro) **+** one 800–1000 VA UPS **+** one spare HDMI cable. That's the whole "server hardware".

> Extra flash drives: buy **two USB flash drives (8 GB or bigger)** — one for moving files during > installation, and one or two for the daily backup (Section 13).

About the UPS (battery box): the server must survive small power cuts by itself. An 800–1000 VA UPS is a box you plug into the wall, then plug the server (and router) into the UPS. If power blips, the UPS keeps them running for 10–30 minutes. This is not optional for a hospital system.

2. What YOU must prepare at your office first (do this before visiting the hospital)

2.1 Build the newest version of the program

Open PowerShell on your development computer and run these **exact** commands:

```
# 1. Build the server part
cd C:\Users\LuckyGold\Desktop\Sretan EMR\server
npm install
npm run build

# 2. Build the website part
cd C:\Users\LuckyGold\Desktop\Sretan EMR\client
npm install
npm run build
```

When both finish without errors, copy these folders onto your **USB flash drive**:

- `server\dist` (the finished server)
- `client\dist` (the finished website)
- `scripts\` (whole folder — contains helper files we will run on the server)

2.2 Make the online "post office" (GitHub) safe for hospitals

1. Open github.com → log in.

2. Settings → Developer settings → **Personal access tokens** → **Fine-grained tokens** → Generate new token.
3. Name it `hospital-readonly`. Choose **Only select repositories** → your EMR repository.
4. Under **Permissions** → **Contents** set **Read-only** (nothing else).
5. Create it. Copy the token that starts with `github_pat_...` and save it somewhere safe (you will paste it on each hospital server once).
6. **Never use your personal GitHub password on a hospital computer.** This token can only READ (download) your code — it can never change it, even if a hospital machine is stolen.

2.3 Make the online "cloud room" (Supabase)

1. Go to supabase.com → create an account → **New project** (name it e.g. `sretan-cloud`, choose a password you save). This becomes your shared **Cloud SaaS** project for all hospitals.
2. After it is ready, open **SQL Editor**.
3. You need the schema (the blueprint) to paste there. Get it from any running copy later, or use the file `database/065_machine_update_reports.sql` + all files before it. Simplest first time: after you finish Section 9 on the first hospital, open in that hospital's browser `http://localhost:3000/api/superadmin/schema-export?inline=1` and copy the `sql` text.
4. Paste it into the Supabase **SQL Editor** and click **Run**.
5. Copy two values from **Settings** → **API**:

> The cloud schema must be **re-run** in Supabase whenever you add new database files later > (Section 16 explains exactly when).

3. First day at the hospital — unpacking and first power-on

3.1 Plug things in

1. Take the server box out of the box.
2. Connect these **temporarily** (you will remove them later):
3. Connect the **Ethernet (network) cable**: one end into the server box, the other end into one of the LAN sockets on the hospital router. (Wired is always better than WiFi for the server.)
4. Plug the **power** of the server into the **UPS**, and the UPS into the wall socket.
5. Switch the server on (press its power button). It will start like a normal computer.

3.2 Finish the first Windows setup (only once)

Windows will ask simple questions — follow them like any new computer:

- Language → keyboard layout → **Next**.
- When asked about network, choose the hospital network (or "Domain join instead" if it appears, choose "Set up for personal use" / next).
- Create TWO accounts when it asks:
- Let Windows finish. You are now looking at the desktop.

3.3 Turn OFF sleeping (so the server never "falls asleep")

1. Press the **Windows key**, type **"Power & sleep settings"** and press Enter.
2. Under **Sleep**, set **On battery / When plugged in** to **Never**.
3. Also set **Screen**: *When plugged in* → **Never** (or a few minutes — it doesn't matter since we remove the screen later).

3.4 Turn ON "come back by itself after a power cut" (BIOS setting)

1. Restart the computer. While it is restarting, tap the **F2** or **Delete** key repeatedly until a blue/grey screen (the BIOS) appears. (Which key depends on brand — Dell = F2, HP = F10 or Esc.)
2. Look for a setting named **"Restore on AC Power Loss"**, **"AC Power Recovery"**, **"After Power Loss"**, or similar (often under *Power Management*).
3. Set it to **Power On / Last State** (choose **Power On**).
4. Press **F10** → **Save and Exit** (or follow the on-screen instruction).
5. Windows will start normally again.

Result: if the building power goes off and comes back, this computer turns itself back on with no human help.

3.5 Stop Windows from restarting during the day

1. Windows key → type **"Windows Update settings"** → Enter.
2. Click **Advanced options** → **Active hours**.
3. Set active hours e.g. **07:00 – 22:00**. Windows will not restart during those hours.

3.6 Set the power plan to High Performance

1. Windows key → type **"Choose a power plan"** → Enter.
2. Choose **High performance** (if you only see "Balanced", click "Show additional plans").

3.7 Turn on Remote Desktop (so you can control it from your laptop later)

1. Windows key → type **"Remote desktop settings"** → Enter.
2. Turn **Remote Desktop** ON. Note the computer name shown there (write it down).
3. Optional but recommended: install **RustDesk** (free, rustdesk.com). Run it on the server once and set a **permanent password**. Note the **ID**. From now on you can reach this server from anywhere.

3.8 Find the server's address on the network

1. Windows key → type **cmd** → press Enter (a black window opens).
2. Type `ipconfig` and press Enter.
3. Look for the **IPv4 Address** (e.g. `192.168.1.200`). **Write this number down** — this is the server's address that every clinic computer will use. From now on I will call it `HOST-IP`.

4. Install the three programs the server needs

Open the browser (Edge/Chrome) and download/install these, in this order. Click "Yes" on the "allow changes" popups and accept defaults unless told otherwise:

4.1 Node.js (the engine that runs the program)

1. Go to nodejs.org → download the **LTS** version → install → **Next** all the way → Finish.

4.2 PostgreSQL (the filing cabinet that stores data)

1. Go to postgresql.org → download **Windows x86-64** of version 16 or 18 → run the installer.
2. Click Next until it asks for a **password**. **Type a password you choose now and WRITE IT DOWN** (example: `Hospital@2026`). This password is very important.
3. Keep the default port **5432**. Finish the installer.

4.3 Git (the program that downloads your updates)

1. Go to git-scm.com → download Windows → install with default options.
2. During install it may ask about "default branch name" — choose **master** if asked.
3. It may ask about "credential manager" — keep the default.

4.4 NSSM (the program that keeps the server always on)

1. Go to nssm.cc → download the **latest zip** → extract it.
2. Copy the file `nssm.exe` (from the `win64` folder) into a folder you created: `C:\nssm\`.

4.5 Test tools you will use during setup

- **Chrome** (optional, for testing the website).
- Keep the black **cmd/PowerShell** window method in mind — we will use it a lot below.

5. Put the program on the server (copy the code)

5.1 Download your code from GitHub onto the server (once)

1. Plug your **USB flash drive** (from Section 2.1) into the server.
2. Press **Windows key**, type **PowerShell**, right-click it and choose **Run as administrator**.
3. Type this and press Enter (this creates the main folder):

```
New-Item -ItemType Directory -Path C:\hms
```

1. Download the code into it. Run:

```
git clone https://github.com/LuckyGoldx/sretan-hms.git C:\hms
```

(GitHub may ask for a login the first time — you can use the read-only token from Section 2.2 as the password, or finish later with the setup script in Section 10.4 which stores it safely.)

5.2 Put the "finished" website and server files in place

The downloaded code is the *recipe*. The server runs *finished dishes* (`dist`), which are not inside the download. So copy them from your USB:

```
# Copy from your USB drive (change E: to the letter of your flash drive)
Copy-Item -Recurse -Force E:\dist\server\*   C:\hms\server\dist\
Copy-Item -Recurse -Force E:\dist\client\*   C:\hms\client\dist\
```

(Check the folder exists now: `C:\hms\server\dist\server.js` and `C:\hms\client\dist\index.html`.)

5.3 Install the parts the server needs to run and rebuild

```
cd C:\hms\server
npm install

cd C:\hms\client
npm install
```

(First time takes a few minutes. This installs *everything*, including the tools that let the server rebuild itself later when you push updates — that is what makes remote updates work.)

6. Create the database and tell the server its password

6.1 Create the empty database

```
& "C:\Program Files\PostgreSQL\18\bin\createdb.exe" -U postgres sretan_emr
```

- It will ask for the PostgreSQL password you chose in Section 4.2. Type it.
- If you installed a different PostgreSQL version (16, 17), change the `\18\` part to your version.
- If you get "already exists", that is fine — move on.

6.2 Tell the server the database password (environment variables)

1. Windows key → type **"Edit environment variables for your account"** → Enter.
2. Click **Environment Variables…** (bottom).
3. Under **System variables**, click **New** and add each of these (one by one, exact spelling):

Variable name	Value
`PG_PASSWORD`	the PostgreSQL password from Section 4.2
`PORT`	`3000`

(Leave the others unset — they already have sensible defaults.)

Click OK everywhere to close.

7. First test run (prove it works before making it automatic)

1. Open **PowerShell as administrator**.
2. Run:

```
cd C:\hms\server
node dist\server.js
```

1. Watch the text. You should see lines like `Running migration: 001_...`, `... 065_...`, and finally **`MACHOKO HMS Server running on port 3000`**. The database was created automatically by those migrations — you do nothing extra.
2. Open a browser on the server and go to `http://localhost:3000`. You should see the **login page**.
3. Also run this to double-check the "brain" responds:

```
Invoke-RestMethod -Uri "http://localhost:3000/api/health"
```

You should see `{status: ok ...}`.

1. Press **Ctrl + C** in the PowerShell window to stop it (we will now make it run forever instead).

> If `http://localhost:3000` shows nothing or an error, the `dist` folders from Section 5.2 are > missing or outdated — copy them again and re-run step 2.

8. Make the server "always on" (a Windows Service)

A **Windows Service** is a program that Windows starts by itself every time the computer turns on, with no one logging in, and restarts automatically if it crashes.

1. Open **PowerShell as administrator**.
2. Run these commands one by one (this registers the program as a service named `Hospital_EMR_Local_Core`):

```
C:\nssm\nssm.exe install Hospital_EMR_Local_Core "C:\Program Files\nodejs\node.exe"
"C:\hms\server\dist\server.js"
C:\nssm\nssm.exe set Hospital_EMR_Local_Core AppDirectory "C:\hms\server"
C:\nssm\nssm.exe set Hospital_EMR_Local_Core Start SERVICE_AUTO_START
C:\nssm\nssm.exe set Hospital_EMR_Local_Core AppStdout "C:\hms\logs\server_runtime.log"
C:\nssm\nssm.exe set Hospital_EMR_Local_Core AppStderr "C:\hms\logs\server_runtime.log"
C:\nssm\nssm.exe set Hospital_EMR_Local_Core AppRestartDelay 10000
net start Hospital_EMR_Local_Core
```

1. Make it start a few seconds AFTER the database (so the order is always correct):

```
sc.exe config Hospital_EMR_Local_Core start= delayed-auto
```

1. **Check it is running:** press Windows key, type **services.msc**, Enter. Look for **Hospital_EMR_Local_Core** → Status should be **Running**, Startup Type **Automatic (Delayed Start)**. Also check **postgresql-x64-18** (or your version) is Running/Automatic.

1. **Test the always-on behaviour now:** right-click `Hospital_EMR_Local_Core` → **Restart**. Wait 15 seconds. Open `http://localhost:3000` — it works again by itself. That is exactly what happens after any power cut or reboot. No human needed.

> Every time Windows starts: PostgreSQL starts → (delayed) the EMR service starts → the clinic is live. > This is the whole "always on" secret.

9. Tell the firewall to allow the clinic computers in

Windows has a guard (the **firewall**) that blocks other computers by default. Open just the one door (port 3000) on the **private** network:

```
netsh advfirewall firewall add rule name="Sretan EMR API" dir=in action=allow protocol=TCP localport=3000
profile=private
```

Fix the server's address so it never changes (choose ONE):

- **Easiest — ask the router:** log into the router (usually a web page at `192.168.1.1` or printed on the router) → find **DHCP Reservation / Address Reservation** → add the server's MAC address (find it with `getmac` in PowerShell) → give it the IP you wrote in Section 3.8 → Save.
- **Or set it on Windows:** Settings → Network → your connection → IP assignment → **Edit** → **Manual** → **IPv4** → IP: your `HOST-IP`, Mask: `255.255.255.0`, Gateway: `192.168.1.1` → Save.

Make the WiFi router let computers talk to each other: log into the router and look for **"AP isolation"**, **"Client isolation"**, or **"Guest network"** — turn isolation **OFF**. Many routers ship with it ON, which secretly blocks the clinic computers from reaching the server even though they show WiFi bars. (If the router looks unfamiliar, Section 9A below explains the router pages step by step.)

9A. The router and the hospital network — explained step by step

Think of your hospital network like one house:

- **The router** is the front door: it brings the internet in, gives every device its address, and makes the WiFi.
- **The switch** is a plug board: it lets many cables talk to each other. It has no brain of its own — the router is still the boss.
- **The server and all clinic computers** are the people in the house. They all live on the SAME network, so they can all reach each other.

9A.1 How to wire it (hospital has a LAN cable network AND a switch)

1. Take a network cable from one of the router's **LAN sockets** (the numbered ones) into the switch. (If you only have a few computers, you can plug them straight into the router's LAN sockets and skip the switch.)
2. Plug the **server** into the switch (or into the router) with a cable.
3. Plug every **clinic desktop** into the switch with a cable — or connect them to the router's WiFi.
4. Switch everything on. Wait one minute.

Picture:

```
        Internet
           |
        [Router]  ← gives addresses + WiFi
           | cable
        [Switch]  ← the plug board
        /     |     \
  [Server] [Desktop 1] [Desktop 2]
  (192.168.1.200)   ... more desktops ...
```

The one rule that keeps it working: let only ONE router hand out addresses (the hospital router). Do not plug a second router into the network to "extend" it — plug in a switch instead. Two routers would create two different houses, and the computers would not find each other.

How to check everything is in the same house: on each computer, run `ipconfig` (Section 3.8). The IP numbers must all start the same, e.g. `192.168.1.xxx`. If a computer shows something like `169.254.xxx.xxx`, its cable or switch port is bad. If it shows a completely different range (`10.x.x.x` or `192.168.8.x`), it is plugged into a different router — fix the wiring.

9A.2 How to factory-reset a router (Airtel or any brand)

You might need this if the router was used before, has a forgotten password, or someone changed settings and you want a clean start.

1. Find the **small reset button** on the router. It is usually a tiny hole on the back or bottom, sometimes labelled **RESET**. You need a pin or a toothpick to press it.
2. With the router ON and powered, press and **hold the reset button for 10–30 seconds** (keep holding even when the lights go off). Let go only when the lights blink together. The router restarts.
3. Wait 2 minutes. The router is now back to **factory settings** — every old setting is erased (including the old WiFi password and any old AP-isolation setting, which is exactly why a reset can fix "computers can't talk" problems).
4. Connect a computer to the router with a cable. Open a browser.
5. Find the router's page address: open **cmd** → type `ipconfig` → look at **Default Gateway** (often `192.168.1.1` or `192.168.0.1`). Type that into the browser address bar, e.g. `http://192.168.1.1`.
6. Log in. The username/password is printed on a **sticker on the router** (often `admin` / `admin`, or `admin` / `password`). If you changed it before and forgot it, you must reset again.
7. Set up your hospital WiFi: give it a **name (SSID)** and a **strong password**, choose **WPA2 or WPA3** security, and save. Write the name and password down.

Every brand's page looks a little different, but the job is the same. On an **Airtel** router look for tabs like **Wireless / WLAN / WiFi Settings / Advanced**; on others it may be **TP-Link**, **Huawei**, **ZTE**, or **Tenda** — the words are similar. When you cannot find a setting, look for a **search box** in the router page and type the setting name.

9A.3 How to turn OFF "isolation" (so computers can find each other)

"AP isolation / client isolation" is a setting that makes WiFi computers see the internet but NOT each other. If it is ON, the clinic desktops cannot reach the server even though they have full WiFi — this is the most common reason "nothing loads".

1. Log into the router page (Section 9A.2, step 5–6).
2. Go to the **WiFi / Wireless settings** page.
3. Look for one of these names and turn it **OFF** (untick / set to Disabled / slide it off):
4. If the router has more than one WiFi (2.4 GHz and 5 GHz), switch isolation **off for each one**.
5. If the router has a **Guest network**, either turn the guest network OFF, or inside the guest settings turn ON "allow access to the local network". Guest networks are isolated by design.
6. Click **Save / Apply** and wait one minute. Then test from a clinic desktop: `ping HOST-IP` (Section 3.8) should now get an answer.

If you cannot find any "isolation" name, look in the router's **Advanced / Advanced Wireless / Device Management** pages, or type "isolation" into the router's search box. After a factory reset (Section 9A.2), isolation is usually OFF by default.

9A.4 How to make the clinic desktops load the server over the LAN

Everything below needs two things already done: (1) the server is wired into the same router/switch as the desktops, and (2) the server has a **fixed address** (Section 9 — DHCP reservation or static IP). Then each clinic desktop opens the EMR in its browser (Section 12).

9A.5 Using a readable link instead of the IP (e.g. `http://emr-server:3000`)

Typing `http://192.168.1.200:3000` works, but a name is easier for staff: `http://emr-server:3000`. A name is like saving a phone contact — instead of dialling the number every time, you save the name.

Step 1 — give the server a friendly name (do ONCE on the server):

1. On the server: Windows key → type **"Rename your PC"** → press Enter.
2. Click **Rename**, type a short name with no spaces, e.g. **emr-server**, and click **Next**.
3. Click **Restart Now**. (The server comes back by itself — services auto-start, Section 8.)
4. Confirm the address is still the fixed one: run `ipconfig` and check the IP matches Section 3.8.

Step 2 — make each clinic computer understand the name (2 minutes per computer).

Two ways. Try **Option A** first; if a computer can't find the name, use **Option B** for that computer.

Option A — automatic (Windows looks up the name by itself): On each clinic computer, make sure **Network discovery** is on: Settings → Network & Internet → Advanced network settings → Advanced sharing settings → turn ON **Network discovery** and **File and printer sharing** for "Private". Then open `http://emr-server:3000`. If it loads, you are done with that computer.

Option B — the hosts file (reliable, always works): This file is a little phone book that Windows reads before anything else. Add one line that says "when someone says emr-server, go to 192.168.1.200".

1. On the clinic computer, press the Windows key, type **Notepad**.
2. Right-click Notepad → **Run as administrator** → click **Yes**.
3. In Notepad: File → Open. In the "File name" box type exactly: `C:\Windows\System32\drivers\etc\hosts` and press Enter.
4. At the very bottom of the file, add ONE new line (use your real server IP from Section 3.8, then a space or tab, then the name):

```
192.168.1.200  emr-server
```

1. File → Save. (If Windows says it can't save, you did not open Notepad as administrator — do step 2 again.)
2. Close Notepad. Now open `http://emr-server:3000` in the browser — it should load.

Step 3 — bookmark it. On every clinic desktop, open `http://emr-server:3000`, log in, and press **Ctrl + D** to bookmark. Staff only ever click the bookmark.

Important: the readable name only works AFTER the server's IP is fixed (Section 9). If you later change the server's IP, update the hosts line on every computer (or better, keep the fixed IP from Section 9 so you never need to). Always keep

`http://<HOST-IP>:3000` as the fallback address — if a name stops working one day, the IP address still opens the system.

10. Configure the EMR itself (login, hospital, cloud, updates)

10.1 First SuperAdmin login (change the password!)

1. On the server's browser, open `http://localhost:3000/superadmin/login`.
2. Log in with the default **username** `lucky` and **password** `lucky`.
3. **Immediately change this password.** Look in the SuperAdmin pages for where you change your own account/password (SuperAdmin → Staff or Settings). If there is no change screen, you (the developer) change it in the database or ask the EMR to add a change-password screen — treat `lucky` as an emergency-only code.

10.2 Create/select this hospital (each hospital = one tenant)

1. In the left menu click **Hospitals**.
2. If the hospital isn't there, click **Add/Setup Hospital** and fill its name and details.
3. On the hospital's row click **Enter Hospital** (activate it). This tells this server machine: "the hospital you belong to is THIS one". (Your guide calls this the "active hospital".)

10.3 Connect this hospital to the shared cloud (Cloud SaaS)

1. Left menu → **Cloud & Sync**.
2. On the **Deployment** tab, in the box **"Cloud SaaS — Global Provider Credentials"**, paste the **Supabase Project URL** and **anon key** from Section 2.3, then **Save Credentials**.
3. Go to this hospital's **Settings** (Hospitals → this hospital → Settings, or the Setup console) and set its **Deployment mode = Cloud SaaS**. Saving automatically turns cloud sync on.
4. Watch the black server log (or `C:\hms\logs\server_runtime.log`): you should see `Sync cycle starting...` every 15 seconds. The hospital is now "phoning home".

> Why paste the same cloud details on the hospital too? Because each hospital's server reads the > shared cloud project from ITS OWN settings. The hub console (Section 11) does the same for you.

10.4 Give the server your read-only GitHub token (so it can receive updates safely)

Run in **PowerShell as administrator**:

```
powershell -ExecutionPolicy Bypass -File C:\hms\scripts\hospital_git_setup.ps1 `
  -Repo "LuckyGoldx/sretan-hms" -Branch "master" -Token "github_pat_YOUR_TOKEN_HERE"
```

What it does (automatically): sets the download address, stores the token in Windows' secret safe (NOT in the code), blocks the server from ever pushing back, and tests that it can download. You should see `SUCCESS`.

10.5 Turn ON automatic updates on this server

1. Left menu → **Cloud & Sync** → **Software Update**.
2. Under **Git Repository & Security** confirm the repository URL shows your repo (no secret inside) and the branch is `master`. Click **Verify Remote Access** — it should go green.

3. Switch **"Auto-update this machine"** to ON and set the check interval to **5** minutes.
4. Click **Save Auto-Update Settings**, then **Check Now**. You should see *Up to date* and the branch.

10.6 Create the "apply updates" helper (the last piece of remote magic)

New code is downloaded by the step above, but the running program is the "finished" version. A small helper must rebuild and restart. Windows can run it automatically every 10 minutes.

Run in **PowerShell as administrator**:

```
schtasks /create /tn "EMR Apply Updates" /sc minute /mo 10 /ru SYSTEM /f `
  /tr "powershell -ExecutionPolicy Bypass -File C:\hms\scripts\apply_update.ps1"
```

Test it once:

```
powershell -ExecutionPolicy Bypass -File C:\hms\scripts\apply_update.ps1
```

It should finish in under a second (nothing to do yet) with no error.

11. Set up YOUR office hub console (the remote control)

Do this on your own office computer or laptop (the same build from Section 2.1):

1. Install Node.js and PostgreSQL on it too, and create its database:

```
& "C:\Program Files\PostgreSQL\18\bin\createdb.exe" -U postgres sretan_emr
```

1. Put the code on it the same way as Section 5 (clone + copy `dist` + `npm install`).
2. Start it when you need it:

```
cd C:\hms\server
node dist\server.js
```

1. Open `http://localhost:3000/superadmin/login`, log in (and change `lucky`).
2. **Cloud & Sync** → **Deployment**: paste the SAME Supabase URL + anon key, save.
3. Open **Fleet Monitor** (left menu). Once the hospital from Section 10 has run for a few minutes, its row appears here: hospital name, machine, branch, applied commit, "Up to date" badge.

This console is now your **remote control panel** for every Cloud SaaS hospital.

12. Connect every clinic computer (the "TVs")

On each hospital desktop (records desk, doctors, pharmacy, lab, paypoint, maternity…):

1. Open Chrome or Edge.
2. Type `http://HOST-IP:3000` (use the number from Section 3.8, e.g. `http://192.168.1.200:3000`).
3. Log in with the staff account (e.g. doctor/nurse/pharmacy accounts you created for this hospital).
4. Press **Ctrl+D** to bookmark it.

That is the entire "installation" on a desktop. Desktops never receive the program — they only look at the server. Nothing to update on them, ever.

Optional nice touch: put a shortcut on each desktop. Right-click desktop → New → Shortcut → for location type: `chrome.exe --kiosk http://HOST-IP:3000` (full path if needed), name it "Hospital System".

13. Daily safety copy (flash backup) — today, do it by hand

What is missing right now: the automatic "backup to USB every night" feature is planned but not built yet. Until it exists, do this **once a day** (30 seconds):

1. On the hospital server's browser, go to SuperAdmin → **Backup & Restore**.
2. Click **Create Full Backup**. Wait for it to finish.
3. Click **Download** and save the `.sbackup` file onto the backup flash drive.
4. Unplug the flash drive and keep it safe (ideally alternate two drives; keep one offsite).

Restore (if the server dies): set up a fresh server using Sections 3–8, then SuperAdmin → Backup & Restore → **Restore** → choose the `.sbackup` from the flash. The hospital is back.

14. Test the whole circle once (do this before you leave the hospital)

1. From your office hub console, open **Fleet Monitor**. Confirm the hospital shows **Up to date**.
2. From the hub: **Cloud & Sync** → **Software Update** → **Publish Update**. (You can also test the targeted button **Roll out** on the hospital's row in Fleet Monitor.)
3. Watch Fleet Monitor refresh: within a minute the hospital's commit should still show up to date (or advance after you make a change in Section 15). The **EMR Apply Updates** task rebuilds and restarts within 10 minutes, then the hospital is running the newest version.
4. Ask a staff member to open the website on a clinic desktop and confirm it works.

15. Your daily work: pushing updates from your office

15.1 Sending NEW CODE to all hospitals (or just one)

Option A — just push (simplest):

```
cd C:\Users\LuckyGold\Desktop\Sretan EMR
git add -A
git commit -m "describe your change"
git push origin master
```

Every hospital with auto-update ON checks the download address every 5 minutes, sees new code, and downloads it. The apply helper rebuilds + restarts within ~10 minutes. Applied automatically.

Option B — push AND ring the bell (faster, ~15 seconds to download):

1. Do the push above.
2. Hub console → **Cloud & Sync** → **Software Update** → **Publish Update** (this "rings the bell" for ALL hospitals over the cloud).

Option C — send to only ONE hospital (tenant):

1. Do the push above.
2. Hub console → **Fleet Monitor** → find that hospital → click **Roll out**.
3. Only that hospital's server reacts. Others ignore it.

Check it worked: Fleet Monitor shows the hospital's **Applied commit** = your new code and badge **Up to date**, last pull **OK**.

> Reminder — the hospital only receives this if: (1) its internet is on, (2) auto-update is ON, > (3) the apply task from Section 10.6 exists. Offline hospitals catch up automatically the next time > they are online.

15.2 Making DATABASE changes remotely

A database change is a new file like `database\066_xxx.sql` that you add to the code (see the existing files for the exact style — they must be numbered and safe to run again).

1. Add the new file, then **push + ring the bell** (Section 15.1).
2. When a hospital downloads and restarts, it runs that database change on its OWN computer automatically. The clinic keeps working — its local database is updated by itself.
3. **You** must update the cloud copy too, or the hospital's data cannot travel up. In the Supabase project: **SQL Editor** → paste the current schema (get it from any updated machine at `http://localhost:3000/api/superadmin/schema-export?inline=1`, or the Cloud page **Copy SQL**) → **Run**.
4. Watch Fleet Monitor / sync logs — data flow resumes.

Golden rule: ship code and its database change together; never make a database change that breaks old code; always re-run the schema in Supabase after adding migration files.

16. What happens in everyday situations (so you are never surprised)

Situation	What happens	Do you need to do anything?
A clinic desktop is switched off / on	Nothing lost — data lives on the server	No. Open the bookmark after restart
The WiFi router restarts	Clinic computers drop for ~1 minute, then reconnect by themselves	No. Refresh the page
Hospital power cuts out and returns	Server box (on UPS + BIOS auto-on) boots itself → PostgreSQL starts → EMR service starts	No, if you did Sections 3.3–3.6 and 8. Unsaved typing is lost
A hospital has no internet for a day	Clinic works 100% normally	No. Updates/sync wait and resume by themselves
You push code while a hospital is offline	Nothing happens yet	No — the hospital applies it when it is next online

17. Honest list: what is missing / gaps today

1. **Automatic flash backup** — not yet built. Use the manual daily download (Section 13). This is the top "nice to have" to automate next.
2. **Remote "switch off auto-update" button** — the auto-update switch lives on each hospital server. You cannot turn it on/off from the hub yet (deliberate safety feature, but it means a hospital that switched it off must be touched or told how).
3. **No instant code activation** — downloads are fast, but activating new code needs a rebuild + restart, which happens on the apply task (≤10 min). You can shorten by making both intervals smaller (e.g. auto-update check 1 min, apply task every 2 min).
4. **One hospital = one server machine** — the system supports several hospitals on one machine for data, but remote updates follow the *active* hospital of that machine. Keep one hospital per box.
5. **Private Cloud hospitals** — fully supported, but each uses its OWN Supabase project. To manage one from the hub, the hub's active hospital must match that hospital. The "see ALL hospitals in one Fleet Monitor" view works for **Cloud SaaS** (shared project) — which is what this guide sets up.
6. **Supabase permissions** — keep the cloud project's settings exactly as the app expects (it syncs with the public anon key). Do not tighten them yourself or sync/fleet silently stop.
7. **SuperAdmin password change screen** — default `lucky/lucky` must be changed; if the console has no "change password" page yet, that is a small missing feature to add (or change it in the database).
8. **Remote desktop on the server** (Section 3.7) is your safety net for anything the system can't do by itself (e.g., fixing a machine that won't boot).

18. Quick daily/weekly habits

Hospital staff:

- Never switch off / restart the server box. Never unplug it. Never plug anything into it except the backup flash drive. (Put a printed note on the box.)
- Plug the backup flash drive in at closing; remove it in the morning.

You (developer/operator), weekly:

- Open the hub → Fleet Monitor: all hospitals "Up to date", last pull OK.
- Check the Supabase project has enough space and the hospital rows are syncing.
- Confirm at least one flash backup exists per hospital.

19. Final one-page checklist (server)

- [ ] Server box: business desktop, i5, 16 GB RAM, 512 GB SSD, Windows 11 Pro + UPS
- [ ] Windows: sleep OFF, BIOS "restore on power loss" ON, active hours set, High Performance plan
- [ ] Remote Desktop + RustDesk ON (so you can reach it from anywhere)
- [ ] Node.js, PostgreSQL, Git, NSSM installed
- [ ] `C:\hms` = git clone + `server\dist` + `client\dist` overlaid + `npm install` done twice
- [ ] Database `sretan_emr` created; `PG_PASSWORD` and `PORT` environment variables set

- [ ] First boot test: `http://localhost:3000` shows login page
- [ ] Service `Hospital_EMR_Local_Core` = Automatic (Delayed Start) + Running; tested "Restart"
- [ ] Firewall rule for port 3000 added; server IP reserved on router; AP isolation OFF
- [ ] SuperAdmin logged in and `lucky` password changed
- [ ] Hospital tenant created + activated (Enter Hospital)
- [ ] Cloud & Sync: Supabase URL + anon key saved; deployment mode = Cloud SaaS
- [ ] `hospital_git_setup.ps1` run; **Verify Remote Access** green
- [ ] Auto-update ON (5 min), branch master
- [ ] Scheduled task **EMR Apply Updates** created and test-run
- [ ] All clinic desktops open `http://HOST-IP:3000`
- [ ] From the hub: hospital appears in Fleet Monitor; a test Publish/Roll out works
- [ ] Flash backup taken today

20. Moving to a NEW server (change the machine)

One day you may need to swap the hospital's server box for another one — because the old one broke, you bought a newer/faster machine, or you want to move from a test machine to the real one. There are two ways, and you choose based on ONE question:

> **"Do I want the new server to start with ALL the old data (patients, staff, history), or start fresh > with nothing?"**

- **WITH data** → follow **Case A** below (uses the backup file).
- **WITHOUT data (fresh start)** → follow **Case B** below.

There is no third option. Decide first, then follow only that case.

The golden rule before ANY move

Never switch the old server off until the new one is fully working. Run both at the same time for a while if you can — the old one keeps the hospital safe while you test the new one. When the new one is confirmed working, THEN you stop the old one. This is the "test before you jump" rule.

CASE A — Move WITH all data (patients, history, settings)

Step A1. On the OLD server: make a backup file

1. Open the browser on the old server → `http://localhost:3000/superadmin/login` → log in.
2. Left menu → **Backup & Restore**.
3. Click **Create Full Backup** and wait until it says it finished (usually seconds).
4. Find your backup in the list and click **Download**.
5. Save the downloaded file (it ends in `.sbackup`) onto your **flash drive**. Keep the flash drive safe — it is your whole hospital in one small file.

> What is inside the `.sbackup`? EVERYTHING: the patient data, staff, settings, the hospital's name, > the cloud connection details, and even the logos. The new machine will look exactly like the old one.

Step A2. On the OLD server: also write down two numbers (5 minutes)

1. Write down the old server's **IP address** (PowerShell → `ipconfig` → IPv4 Address). You will probably want the new server to use this SAME number (Step A6).
2. Write down the old server's **Windows Administrator username and password** (not strictly needed for the move, but useful if you keep the old machine as a spare).

Step A3. On the NEW server: do a normal first install — but STOP before creating any hospital

The new machine needs everything from earlier in this guide. Do these sections normally:

- Section 3 (unpacking, Windows, sleep OFF, BIOS power-on, Remote Desktop, find the IP)
- Section 4 (install Node.js, PostgreSQL, Git, NSSM)
- Section 5 (put the program on the server: clone the code + copy `dist` + `npm install`)
- Section 6 (create the database and set the password variables)
- Section 7 (first test run — `http://localhost:3000` must show the login page)
- Section 8 (make it a service — NSSM + `net start`)

STOP. Do NOT do Section 10 on the new server. (Do not log in and start configuring a hospital — the backup will bring the hospital with it. You only configure through the backup.)

Step A4. On the NEW server: restore the backup file

1. Open the browser on the new server → `http://localhost:3000/superadmin/login` → log in with the same SuperAdmin details you used on the old server (the backup restores those logins too; if the new install created a fresh one, use that one for now — the restore overwrites it with the old ones).
2. Left menu → **Backup & Restore**.
3. Click **Restore** → choose the `.sbackup` file from your flash drive (or upload it).
4. Click to start the restore and **wait until it says it finished** — do not switch anything off while it is working.

> If the old and new machines run different versions of the program, always restore on the NEWEST > version (the newest program can read an older backup). Do the reverse only if you know what you are > doing.

Step A5. On the NEW server: re-do the "machine-only" steps

The backup brings the DATA, but a few things belong to the machine itself, not the data. Redo them now:

1. **GitHub token** (new machine needs its own safe key): run the helper again:

```
powershell -ExecutionPolicy Bypass -File C:\hms\scripts\hospital_git_setup.ps1 `
  -Repo "LuckyGoldx/sretan-hms" -Branch "master" -Token "github_pat_YOUR_TOKEN_HERE"
```

1. **Auto-update helper task** (new machine): run this again:

```
schtasks /create /tn "EMR Apply Updates" /sc minute /mo 10 /ru SYSTEM /f `
  /tr "powershell -ExecutionPolicy Bypass -File C:\hms\scripts\apply_update.ps1"
```

1. Open SuperAdmin → **Cloud & Sync** → **Software Update** and confirm: repository URL correct, **Verify Remote Access** green, **Auto-update ON**, interval 5 minutes.
2. Check the app's cloud connection is still good: **Cloud & Sync** → **Deployment** — the Supabase URL and key should have come with the backup. If the boxes are empty, paste them again and save.

Step A6. Point the clinic computers at the new server (make it painless)

Do ONE of these:

- **Best — give the new server the OLD server's IP address.** Then every clinic computer keeps working with its old bookmark and you change NOTHING on the desktops. How:
- **Or — change the bookmark on every clinic computer.** If you keep a different IP, visit each clinic desktop once and update the bookmark to `http://NEW-IP:3000`.

Step A7. Test the new server BEFORE touching the old one

1. On the new server: `http://localhost:3000` → log in as a staff user that existed on the old server.
2. Open a patient and confirm history/photos are there. Make a tiny test entry (e.g. register a test patient) — it should save.
3. From your office hub: **Fleet Monitor** should show the hospital and its new machine report.
4. Only when all of this works, go to Step A8.

Step A8. Deactivate the OLD server (only now)

The old machine is no longer the hospital's server. Turn it off properly:

1. On the old server, open **services.msc** (Windows key → type `services.msc` → Enter).
2. Find **Hospital_EMR_Local_Core** → right-click → **Stop**. (Also stop PostgreSQL the same way if you want it fully silent.)
3. To make sure it never starts again on its own: right-click **Hospital_EMR_Local_Core** → **Properties** → Startup type → **Disabled** → OK.
4. You can now unplug the old server and keep it safe as a spare for at least a month. Do NOT delete its files or flash backups until you are sure the new server has been working for weeks.

> Command version of stopping it, if you prefer: > ``` `powershell > net stop Hospital_EMR_Local_Core > sc.exe config Hospital_EMR_Local_Core start= disabled >` ```

Case A — finished ✅

The hospital is running on the new machine with ALL its data. The old machine is safely off.

CASE B — Move WITHOUT data (fresh start on the new server)

Use this when you want a clean slate (new hospital, test machine, or you do not need the old records).

Step B1. On the NEW server: do a complete normal install

Follow every section of this guide as if the new machine were the very first server:

- Sections 3–8: Windows settings, programs, code, database, first test, service, firewall, fixed IP.
- Section 10: log into SuperAdmin, **change the `lucky` password**, create/activate the hospital tenant, set **Cloud SaaS** + Supabase details, run `hospital_git_setup.ps1`, turn **Auto-update ON**, create the **EMR Apply Updates** task.
- Section 12: point the clinic computers at the new server (bookmark or same-IP trick from Step A6).

Step B2. Turn off the OLD server

Only after the new server works (test like Step A7, but expect empty data — that is correct for a fresh start):

1. On the old server: `services.msc` → stop **Hospital_EMR_Local_Core** (and PostgreSQL) → right-click → Properties → Startup type → **Disabled**.
2. Unplug it and keep it safe.

> **A warning for Case B:** starting fresh means the old data is NOT carried over. If you think you > might need the old data later, do Step A1 anyway (make one `.sbackup` from the old server) and keep > that flash drive safe before you switch anything off. You can then restore it later if you change > your mind.

Case B — finished ✅

The hospital is running on the new machine with a clean, empty start.

Quick decision table

Your situation	Do this
Old server broke / upgrading, MUST keep all records	**Case A**
New machine only for testing, or hospital truly starts fresh	**Case B**
Not sure — keep old data just in case	**Case A** (or at least make a `.sbackup` first)

21. Where to get help mid-process

- This guide's sister documents in the project: `DEPLOYMENT_GUIDE.md` (technical reference) and `DEPLOY_TO_HOSPITAL.md` (condensed runbook for you, the developer).
- If a step above behaves differently than written, stop and write down the exact error text — that text is what fixes problems fastest.